Andrew Kozlevcar

11 - 11:50

Question 1

In [1]:
```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    StandardScaler,
    PolynomialFeatures,
    SplineTransformer
)
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.base import clone

# Bike Sharing daily data
url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/00275/"
    "Bike-Sharing-Dataset.zip"
)

import io
import zipfile
import requests

response = requests.get(url)

with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    bike = pd.read_csv(z.open("day.csv"))

bike = bike[["temp", "cnt"]].dropna().copy()

print("Number of days:", len(bike))
display(bike.head())

fig = px.scatter(
    bike,
    x="temp",
    y="cnt",
    labels={
        "temp": "Normalized Temperature",
```

```
        "cnt": "Daily Bike Rentals"
    },
    title="Temperature and Daily Bike Rentals"
)

fig.show()
```

```
Number of days: 731
```

	temp	cnt
0	0.344167	985
1	0.363478	801
2	0.196364	1349
3	0.200000	1562
4	0.226957	1600

Question 1

In [2]:
```
X = bike[["temp"]]
y = bike["cnt"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=123
)

# Linear model
lin_model = Pipeline([
    ("scaler", StandardScaler()),
    ("linreg", LinearRegression())
])

# Degree-9 polynomial model
poly9_model = Pipeline([
    ("scale_x", StandardScaler()),
    ("poly", PolynomialFeatures(
        degree=9,
        include_bias=False
    )),
    ("scaler", StandardScaler()),
    ("linreg", LinearRegression())
])

# Cubic spline with 7 knots
spline_model = Pipeline([
    ("scale_x", StandardScaler()),
    ("spline", SplineTransformer(
        n_knots=7,
```

```
        degree=3,
        include_bias=False
    )),
    ("scaler", StandardScaler()),
    ("linreg", LinearRegression())
])

# Fit all three models
lin_model.fit(X_train, y_train)
poly9_model.fit(X_train, y_train)
spline_model.fit(X_train, y_train)

models = {
    "Linear": lin_model,
    "Polynomial 9": poly9_model,
    "Spline (7 knots)": spline_model
}

# Create a grid for fitted curves
temp_grid = pd.DataFrame({
    "temp": np.linspace(
        X["temp"].min(),
        X["temp"].max(),
        200
    )
})

fig = px.scatter(
    bike,
    x="temp",
    y="cnt",
    opacity=0.5,
    labels={
        "temp": "Normalized Temperature",
        "cnt": "Daily Bike Rentals"
    },
    title="Comparing Three Models"
)

for name, model in models.items():
    fig.add_trace(
        go.Scatter(
            x=temp_grid["temp"],
            y=model.predict(temp_grid),
            mode="lines",
            name=name
        )
    )

fig.show()
```

Part a

From the original visual, we see that the direction of the relationship between temperature and bike rentals in the scatterplot is generally positive; as temperature increases, bike rentals tend to increase. However, this summary obscures the fact that there are complicated sub-shapes in the data. From the part 0 visual (which spaces our axis more nicely), normalized temperatures below 0.5 exhibit a positive linear relationship (though I would not rule out some concave up curvature), whereas temperatures above 0.5 exhibit a negative and non-linear relationship (there is clear concave down curvature in this region).

Part B

As we allow the models to become more flexible, they can demonstrate increasingly heterogenous local function behavior. For the linear model, a constant slope means its behavior is uniform across all intervals; only two points are needed to dictate all of its behavior, too. A degree-9 polynomial allows the slope and curvature to vary, but because it is a global function, 10 points on the curve can dictate its behavior for the remaining points. Similarly, a cubic spline allows for varied curvature and slope, and while any region cannot vary as much as the same region in the degree 9 polynomial, its piecewise structure effectively reduces global homogeneity more; adjacent regions are only dependent up to satisfying continuity and possibly smoothness, depending on implementation. The net effect is that many more than 10 non-arbitrary points are needed to dictate the cubic spline's complete behavior, though its hard to directly compare flexibility of these two models. The non-rigorous point is that as we increase flexibility, local regions are able to demonstrate increasingly exotic behavior that need not align with behavior in other regions. When we compare the polynomial and spline behavior, we see that they largely agree over the intermediate ranges of the data; there is obviously some discrepancy (and who predicts higher depends on the interval), but they remain close. However, at values below 0.2 and above 0.7, the two functions disagree strongly in prediction and form. In the lower region, the 9 degree polynomial constructs a concave down curve with a local maximum that sits far above the spline; the spline, instead, produces a shape that could be reasonably approximated by a line. Similarly, in the region above 0.7, the degree 9 polynomial appears to construct a cocave up curve with a local minimum that sits comfortably above the spline; the spline seems to have a knot in this region, which results in a concave up segment followed by a virtually horizontally tangent line.

Part C

In many cases, we would expect a flexible model to achieve training error that is no larger than that of a less flexible model because the features of the less flexible model are a subset of the features of the more flexible model; in those cases, we are

mathematically guaranteed that the SSE minimization can do no worse because the more flexible model represents an expansion of the feasible set. In the general cases where flexibility is not nested, this does not hold in general but often it does empirically. The reasoning is intuitive. With more flexibility, models have more "levers" or "independent directions" at their disposal, which allow them to move close to a point without sacrificing closeness at other points. Greater flexibility does not neccesarily imply better prediction on new observations. Indeed, we can think of our observation data containing two forms of variation: noise and signal. To predict well on new observations, we want to capture the most signal while simultaneously maximizing the ratio of signal to noise; this means that our prediction model will base most of its assesment on the true underlying patterns of the population. The problem is that increased flexibility often results in more variation being captured, but there is no guarantee that this additional variation being captured is signal; this can result in a lower signal to noise ratio, meaning our predictions are trying to wrongly generalize individual fluctuations to the population.

Question 2

In [3]:
```
rows = []

for name, model in models.items():

    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    train_rmse = np.sqrt(
        mean_squared_error(y_train, train_pred)
    )

    test_rmse = np.sqrt(
        mean_squared_error(y_test, test_pred)
    )

    rows.append({
        "Model": name,
        "Training RMSE": train_rmse,
        "Test RMSE": test_rmse
    })

rmse_table = pd.DataFrame(rows)

display(rmse_table.round(2))
```

	Model	Training RMSE	Test RMSE
0	Linear	1477.19	1628.52
1	Polynomial 9	1386.75	1526.17
2	Spline (7 knots)	1388.46	1513.27

Part A

The training and test RMSE for the linear model is 1477.19 and 1628.52 bikes, respectively. The training and test RMSE for the degree 9 polynomial model is 1386.75 and 1526.17 bikes, respectively. The training and test RMSE for the spline mode is 1388.46 and 1513.27 bikes, respectively. The model with the smallest training RMSE is the degree nine polynomial, whereas the model with the smallest test RMSE is the spline with 7 knots.

Part B

Obviously, decreasing test RMSE does not neccesarily imply decreasing test error. As discussed in 1c, more flexible models tend to capture more variation in the observed data, which translates to lower training RMSE. To predict well on new observations, though, we do not want to maximize the total captured variation; we want to capture the most signal while simultaneously maximizing the ratio of signal to noise. This means that our prediction model will base most of its assesment on the true underlying patterns of the population. The point is that capturing more variation in the training data may end up reducing the signal to noise ratio, leading to worse prediction (and test RMSE). In this case, we see that while the degree nine polynomial had smaller training RMSE than the cubic spline, it actually had a greater test RMSE than the degree nine polynomial.

Part C

As compared to the more flexible models, the linear model has both the highest training RMSE and test RMSE. The highest training RMSE suggests that the model is capturing far less of the variation in the observed training data, and the highest test RMSE suggests that it is likely not capturing enough signal as compared to the more flexible models. However, it is generally hard to say if the other two models are overfitting; note that test RMSE being larger than training RMSE is expected and not reflective of overfitting. Heuristically,overfitting is best gauged by identifying models with low training RMSE but relatively high test RMSE; "relatively" requires benchmark comparisons with other models. In this case, both flexible models have the lowest training RMSE (and therefore candidates for overfitting) but they also have the lowest test RMSEs. While the nine degree polynomial has a slightly larger difference in train and test RMSE, it is hard

to say if the observed differences are just due to noise from a single test-train split. Ultimately, we either need more models of varying complexity to benchmark (especially because the complexity jump from a simple linear model to either of the other two models is quite large), or we need to perform more repetititons (such as cross validation) to be more confident in our observed differences. At this point, I am not confident in concluding whether there is or not overfitting.

Question 3

In [4]:
```
X_new = pd.DataFrame({
    "temp": [0.60]
})

B = 500
rng = np.random.default_rng(123)

y_train_array = np.asarray(y_train)

bootstrap_predictions = []

for name, model in models.items():

    for b in range(B):

        idx = rng.choice(
            len(X_train),
            size=len(X_train),
            replace=True
        )

        boot_model = clone(model)

        boot_model.fit(
            X_train.iloc[idx],
            y_train_array[idx]
        )

        prediction = boot_model.predict(
            X_new
        ).item()

        bootstrap_predictions.append({
            "Model": name,
            "Prediction": prediction
        })

boot_df = pd.DataFrame(
    bootstrap_predictions
)

fig = px.box(
    boot_df,
```

```
    x="Model",
    y="Prediction",
    points=False,
    title=(
        "Bootstrap Predictions at "
        "Temperature = 0.60"
    )
)

fig.show()

variability = (
    boot_df
    .groupby("Model")["Prediction"]
    .agg(["mean", "std"])
    .reset_index()
)

display(
    variability.rename(
        columns={
            "mean": "Mean Prediction",
            "std": "Prediction SD"
        }
    ).round(2)
)
```

	Model	Mean Prediction	Prediction SD
0	Linear	5242.78	69.88
1	Polynomial 9	5838.97	156.05
2	Spline (7 knots)	5976.28	170.01

Here, I will describe the distributions by their summary statistics, and I will then reference these descriptions in the responses that follow. By doing this, I will not bore you with repeating these metrics over and over.

Above, we we have the three bootstrap distributions. The linear distribution is roughly symmetric with a median of $5,239$ bikes, a range of 390 bikes, an IQR of 97 bikes, and a standard deviation of 69.88 bikes. The ninth degree polynomial is roughly symmetric with a median of $5,833$ bikes, a range of 900 bikes, an IQR of 218 bikes, and a standard deviation of 156.05 bikes. Finally, the spline distribution is roughly symmetric with a median of $5,980$ bikes, a range of 1004 bikes, an IQR of 224 bikes, and a standard deviation of 170.01 bikes.

Part A

Based on the above metrics, we can summarize the distributions as following. While all distributions have roughly a symmetric shape, the linear distribution is centered

signficantly lower than the other two distributions, and its spread is also signficantly less (by each of IQR, range, and SD). The ninth degree polynomial is centered quite higher than the linear distribution but a little lower than the spline distribution by about 150. By every observed metric (IQR, range, and SD), its spread is more than the linear distribution but less than the spline distribution (and closer to the latter). Finally, the spline distribution is centered at the highest rental value, and its spread (by every metric) is substantially larger than the linear distribution and the ninth degree polynomial distribution (though much closer to the latter). So, the linear prediction distribution appears most stable and the spline predictio distribution appears most variable.

Part B

By the above metrics, there is a clear ordering of variability across the distributions. For range, IQR, and standard deviation, the linear distribution has the smallest values by a signficant margin. This suggests that, as training samples vary, the linear model's prediction for normalized temperature 0.60 remains consistently around $5,239$ bikes; the model's prediction is relatively insensitive to the sample. Based on the metrics, the ninth degree polynomial has the second largest variability and the spline has the largest, with the ninth degree having a varability much closer to the spline than the linear model. Thus, as training samples vary, both the ninth degree polynomial and the spline prediction for normalized temperature 0.60 fluctuate relatively dramatically around their respective centers of $5,833$ and $5,980$, with the latter varying slightly more; thus, these models' predictions are relatively sensitive to the sample. In general, we see that, with increasing flexibility (as roughly conceptualized through the total number of independent points needed to dictate the model's complete behavior), model predictions become increasingly sensitive to the training sample, which leads to greater prediction variance.

Part C

Our simulation is estimating the bootstrap distribution of our orginal data, which in turn attempts to approximate the true sampling distribution of our model's prediction at normalized temperature 0.60. In this way, if a model's simulated bootstrap distribution exhibits great relative variability (as predictions change substantially across bootstrap samples implies), then this suggests (but does not guarantee) that the true sampling distribution of the model's prediction at normalized temperature 0.60 is also highly variable. Thus, the model's prediction has high variance at this input value. Moreover, the fact that the model fits the training data very closely suggests that the model is capturing a lot of variation in the training sample, but this variation may not be exclusively (or mostly) signal. Thus, the model could be overfitting, but we really cannot make this conclusion without also comparing its Test MSE to other models (through some sort of validation/hold-out strategy). Indeed, depending on the shapes of bias as a function of flexibility and variance as a function of flexibility, it is possible that the true

minimization point of prediction error occurs where there is a lot of variance but almost no bias, and such a model would present like this one. If it is overfitting (meaning the optimization point does not so heavily favor variation), then it will not generalize well; the model will be capturing a lot of noise--perturbations specific to this data--and wrongfully attempt to apply it to the population, where these trends do not exist.

Question 4

In [5]:
```
summary = rmse_table.merge(
    variability,
    on="Model"
)

summary = summary.rename(
    columns={
        "mean": "Mean Bootstrap Prediction",
        "std": "Prediction SD"
    }
)

display(summary.round(2))
```

	Model	Training RMSE	Test RMSE	Mean Bootstrap Prediction	Prediction SD
0	Linear	1477.19	1628.52	5242.78	69.88
1	Polynomial 9	1386.75	1526.17	5838.97	156.05
2	Spline (7 knots)	1388.46	1513.27	5976.28	170.01

Part A

Note that our test and training RMSE compared to prediction distribution are sort of misaligned in scale. Test and training RMSE are about entire model fit while our mean bootstrap predictions and standard deviations are gauging fit on a single prediction point (normalized temperatuee of 0.60). Thus, the comparisons this lab seems to be implying (namely, to extrapolate the behavior at the illustrated point to the rest of the model) are slightly wrong. Nonetheless, I will proceed.

The linear model resulted in the highest training RMSE (1477.19, about 100 bikes higher than the other two models), highest test RMSE (1628.52, about 100 bikes higher than the other two models), but the smallest prediction standard deviation for the bootstrap prediction distribution for normalized temperature 0.60 (69.88 bikes, around 80 to 100 bikes smaller than the other two models). The combined highest training RMSE and test RMSE suggest that the model is capturing relatively little of the variation in our data,

meaning that it is potentially missing out on a lot of genuine structure. Still, its low standard deviation on the prediction bootstrap distribution suggests that its expected prediction error has low variance; this is a pointwise extrapolation. Thus, the estimate of its expected test error (as approximated by the test RMSE) is relatively high, and its expected test error is likely composed of relatively more squared bias because it is oversimplifying the phenomenom and less variance because the model has less capacity to chase noise. Alternatively, the degree-9 polynomial and spline both achieve much lower training (1386.75 and 1388.46) and test RMSEs (1526.17 and 1513.27, respectively); the decreased training RMSE in both cases suggests that the models are capturing relatively more variation from the underlying data, and the fact that the test RMSEs also decreased suggest that this additional variation contains appreciable structure/signal. However, as noted above, the prediction bootstrap distributions for normalized temperature 0.60 are far more variable (standard deviations of 156.05 and 170.01 respectively). Overall, the estimate of the expected test errors of these two models (as approximated by the test RMSE) is relatively low compared to the linear model, and their expected test error is likely composed of relatively more variance (because the models have the capacity to chase more noise) and less squared bias (because they model the underlying phenomenom with a more appropriate level of complexity). Given the similarity in training, test, and bootstrap perforamnce, I do not feel confident distinguishing between the spline and ninth degree polynomial.

Together, these three models capture the bias-variance tradeoff; as we move from the inflexible linear model to the flexible ninth degree polynomials and splines, the squared bias of our model reduces because of a more appropriate representation of our underlying phenomenom. However, the variance increases because the model has more capacity to chase noise. Since the objective, expected test error, is the sum of both error types (and irreducible error), we need to identify an optimal point in which any additional flexibility introduces more variance than it reduces bias, and any additional restriction introduces more bias than it reduces variance.

Part B

As a heuristic, we want a model that generalizes, meaning it performs best on unseen data. To perform well on unseen data, a model must ideally extract all of the signal from the training data while omitting all of the sample-specific noise. A model with the smallest training RMSE suggests that it has captured the most variation in the data. While capturing more variation gives us a greater chance at capturing more signal (and therefore reducing total bias), a wider net also means the chance of capturing more sample-specific noise. The patterns of this noise will not generalize to the actual population, but the model will nonetheless apply it to unseen data; this is known as overfitting, and we leave ourselves vulnerable to overfitting if we choose models solely on their training RMSE. A trademark of overfit models is high prediction variance--large

fluctuations in predictions resulting from small pertubations in the sample; these pertubations do not alter the underlying signal much, but they may drastically shift the observed noise component, and, since an overfit model wrongly absorbs this noise, it too will fluctuate. In reality, we should choose models based on their test RMSE, which better reflects whether the model extracted a sufficient amount of pure signal.

Part C

Tentatively, I would choose the spline model. Moreover, the spline model has the second lowest training and lowest test RMSE of 1388.46 and 1513.27. These marks are significantly lower for than those of the linear model, and its test RMSE is slightly better than the ninth degree polynomial. With the linear model as reference, the spline model captures more of the underlying variation in the observed data (as signified by the improved training RMSE), though it appears that this additional variation captured is net signal (as signified by the improved test RMSE). This choice needs to be considered provisional for several reasons. First, we are attempting to approximate the test RMSE using a single split and with only three models of non-uniform complexity. A single split is a highly variable estimate, and so there is concern that the differences between the two flexible models may be noise; this is particularly concerning when you are considering models who are highly sensitive to the train-test split, which the pointwise analysis of the spline and ninth degree polynomial suggest. Not only is our process for approximating test RMSE noisy, but we are only choosing to test three models, meaning our approximation of the entire test RMSE - flexibility "curve" is limited to just three points. It is hard to say, for instance, whether our spline or ninth degree polynomial is closer to the approximate optimal point of flexibility because we do not have more models to "plot" on this curve we are approximating.